

Revue du diagramme de deployment

Projet Agro IoT - Verification du fichier Diagramme de deployment.drawio

1. Objectif de la revue

Ce document resume les remarques faites sur le diagramme de deployment Draw.io transmis par l'equipe. Le but est de verifier que le schema correspond bien a l'architecture reelle du projet Agro IoT : frontend React, backend Laravel, base de donnees MySQL, utilisateurs par role, securite API et composants IoT.

2. Elements corrects dans le diagramme

- Le diagramme separe globalement l'utilisateur, le backend, la base de donnees, l'ESP32, les capteurs et les actionneurs.
- Il montre une communication HTTP entre le navigateur et le systeme applicatif.
- Il montre une communication SQL entre le backend et la base de donnees.
- Il represente les liaisons GPIO, GPIO relais ou I2C entre l'ESP32, les capteurs et les actionneurs.
- La presence de l'ESP32, des capteurs et des actionneurs est coherente avec le projet IoT.

3. Corrections importantes

Point a corriger	Probleme observe	Correction recommandee
Dashboard Web	Il est place dans le Serveur Backend.	Le placer dans le bloc Frontend React/Vite heberge sur Vercel.
Couche presentation	Elle apparait cote backend.	La presentation appartient au frontend React : Login, Dashboard, Historique, Alertes, Administration, Terminal.
Plateformes de deployment	Vercel, Render et Aiven ne sont pas clairement indiques.	Ajouter Frontend React - Vercel, Backend Laravel - Render, Base MySQL - Aiven.
API Rest	Le libelle est trop general.	Renommer en API REST Laravel et indiquer les routes principales.
Securite API	Le token et les roles ne sont pas visibles.	Ajouter Authorization: Bearer Token, middleware agro.auth et verification des roles.
Acces Donnees (DAO)	Le projet Laravel utilise plutot Eloquent ORM.	Remplacer par Eloquent ORM / Acces base de donnees.
Nombre de capteurs	Le diagramme indique 5 capteurs.	Corriger en 6 capteurs : temperature, humidite air, humidite sol, CO2, luminosite, niveau eau.
HTTP/MQTT	La presence de MQTT n'est pas clarifiee.	Si MQTT est utilise, ajouter un Broker MQTT. Sinon indiquer HTTP/HTTPS ou API ESP32 vers Laravel.

4. Structure de deployment recommandee

La structure suivante correspond mieux au projet actuel :

[Utilisateur : Admin / Technicien / Agriculteur]
| Navigateur Web / PWA
v
[Frontend React + Vite - Vercel]
| HTTPS JSON
| Authorization: Bearer Token
v
[Backend Laravel API REST - Render]
| SQL sécurisée
v
[Base de données MySQL - Aiven]

5. Bloc frontend a représenter

Le frontend doit être un bloc séparé du backend. Il contient l'interface utilisateur et les pages principales.

Bloc	Contenu conseillé
Frontend React/Vite - Vercel	Login, Dashboard, Historique, Alertes, Administration, Terminal, navigation responsive, PWA, Axios.
Flux sortant	Requêtes HTTPS JSON vers Laravel avec Authorization: Bearer Token.

6. Bloc backend a représenter

Le backend doit représenter la logique Laravel et les protections côté serveur.

Bloc	Contenu conseillé
Backend Laravel - Render	API REST, authentification, génération du token, vérification des rôles, logique métier, audit logs.
Sécurité	Middleware agro.auth, vérification Admin / Technicien / Agriculteur, throttle sur login.
Routes principales	/api/auth/login, /api/auth/me, /api/measurements/latest, /api/alerts/active, /api/users, /api/actuators/{actuator}.

7. Bloc base de données

La base de données doit être séparée et clairement nommée selon le fournisseur utilisé.

Element	Details
Base de données MySQL - Aiven	Base actuellement utilisée en ligne. Si le fournisseur change, remplacer Aiven par le nouveau service.
Tables importantes	users, profils, mesures, capteurs, alertes, actionneurs, actions, historiques, seuils, audit_logs.
Flux	Connexion SQL sécurisée depuis Laravel vers MySQL.

8. Bloc IoT

Le diagramme doit clarifier le role de l'ESP32 et la connexion aux capteurs/actionneurs.

Element IoT	Details a afficher
ESP32	Microcontrôleur charge de communiquer avec les capteurs et actionneurs.
Capteurs	Temperature, humidite air, humidite sol, CO2, luminosite, niveau eau.
Actionneurs	Pompe irrigation, ventilateur, lampe/eclairage.
Liaisons physiques	GPIO, GPIO relais, I2C selon les composants.
Communication applicative	HTTP/HTTPS vers Laravel ou MQTT via Broker MQTT si cette option est retenue.

9. Roles utilisateurs a faire apparaitre

Role	Acces principaux
Administrateur	Dashboard, historique, alertes, administration, terminal, gestion utilisateurs, audit, seuils et actionneurs.
Technicien	Dashboard, historique, alertes, terminal, resolution alertes, modification seuils, controle actionneurs.
Agriculteur	Consultation dashboard, historique et alertes. Pas de gestion utilisateurs ni terminal technique.

10. Version textuelle finale a transmettre

Voici une version courte que le collegue peut utiliser pour corriger son diagramme :

Frontend React/Vite - Vercel : Login, Dashboard, Historique, Alertes, Administration, Terminal.
Backend Laravel API REST - Render : Auth, token, roles, audit, logique metier.
Base MySQL - Aiven : users, profils, mesures, alertes, actionneurs, actions, seuils, audit_logs.
Flux Frontend -> Backend : HTTPS JSON avec Authorization: Bearer Token.
Flux Backend -> MySQL : SQL securise.
ESP32 -> capteurs/actionneurs : GPIO, I2C, relais.
Ajouter les roles : Admin, Technicien, Agriculteur.
Corriger 5 capteurs en 6 capteurs.
Clarifier HTTP/MQTT : ajouter Broker MQTT si MQTT est vraiment utilise.

11. Conclusion

Le diagramme actuel est une bonne base, mais il doit etre ajuste pour correspondre au deploiement reel. Les corrections prioritaires sont de separer le frontend du backend, d'ajouter Vercel/Render/Aiven, de représenter le Bearer Token et les roles Laravel, de remplacer DAO par Eloquent ORM et de corriger le nombre de capteurs.